MoonEP: redundant experts, GPU planning, static shapes

How every EP rank receives exactly S×K tokens

Moonshot AI · MoonEP (Kimi K3)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Expert-Parallel Communication · github.com/MoonshotAI/MoonEP

Every GPU must receive exactly S×K token copies

- S tokens per GPU, top-K: S×K copies leave each GPU
- Example: 4 GPUs, 8 experts, S×K = 8
- Benchmark shape: S×K = **65,536**, E/R = **48**

home	GPU 0		GPU 1		GPU 2		GPU 3	
from	E0	E1	E2	E3	E4	E5	E6	E7
GPU 0	3	1	2	1	0	1	0	0
GPU 1	2	1	2	1	1	0	1	0
GPU 2	2	0	2	2	0	1	0	1
GPU 3	2	1	1	1	1	0	1	1
total	**9**	**3**	**7**	**5**	**2**	**2**	**2**	**2**
load	**12**		**12**		**4**		**4**	
– S×K	**+4**		**+4**		**–4**		**–4**	

Offline replicas are re-planned every ten minutes

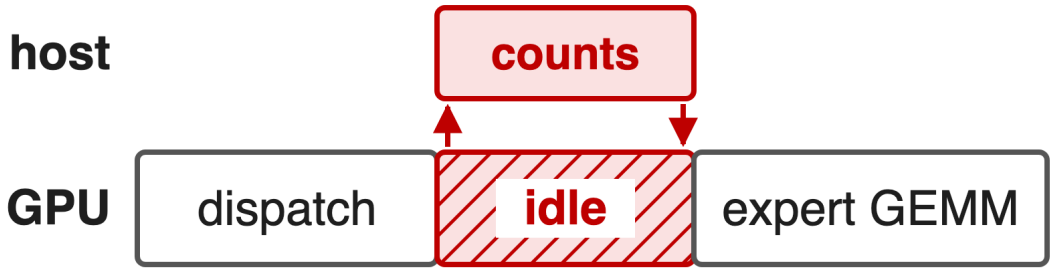
10 min

how often DeepSeek-V3 re-plans its redundant experts from load statistics, while routing changes every micro-batch

DeepSeek-V3 technical report §3.4 (serving deployment); DeepSeek EPLB plans from historical load statistics

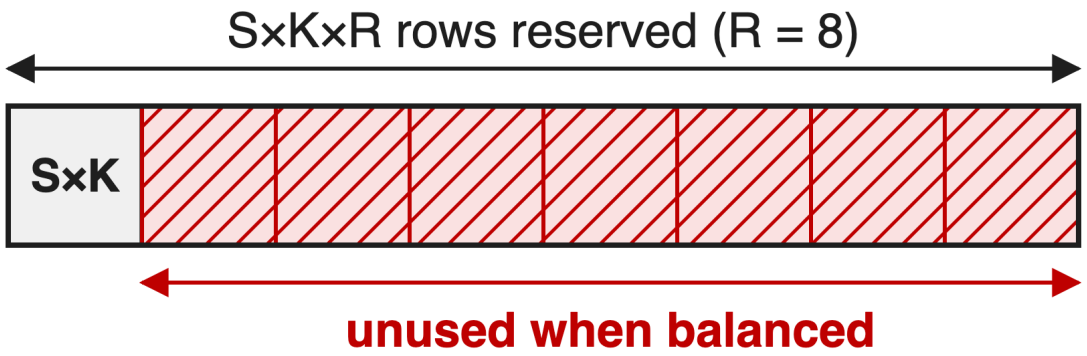
Dynamic counts force a host sync or S×K×R

(a) exact shapes



host sync every layer

(b) worst-case shapes

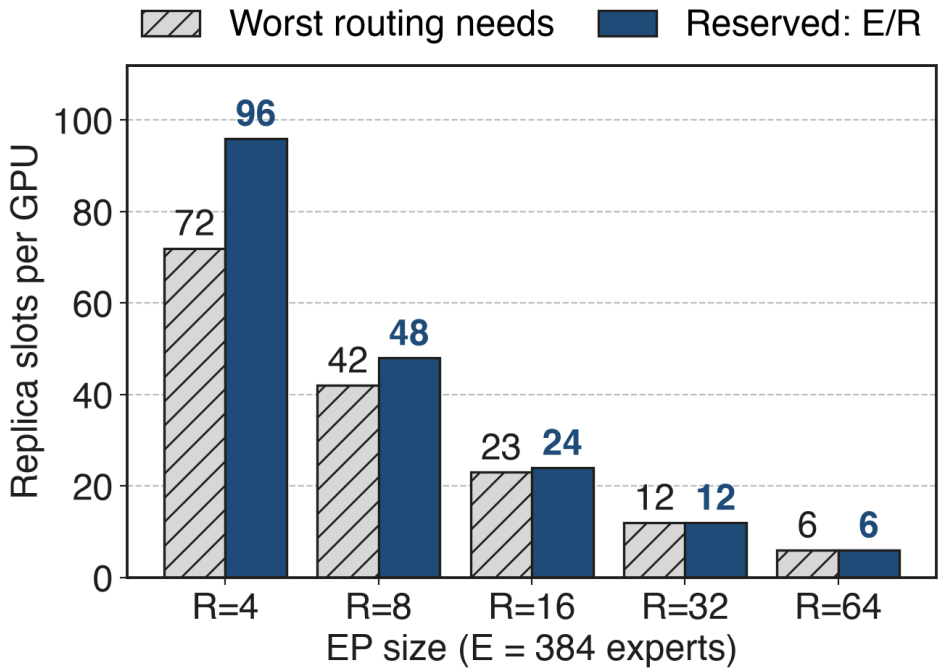


R× buffer memory

Per-step replicas need a fast GPU planner fused with dispatch; DeepSeek-V3 deferred them

Insight: one-source fills need ≤ E/R replicas

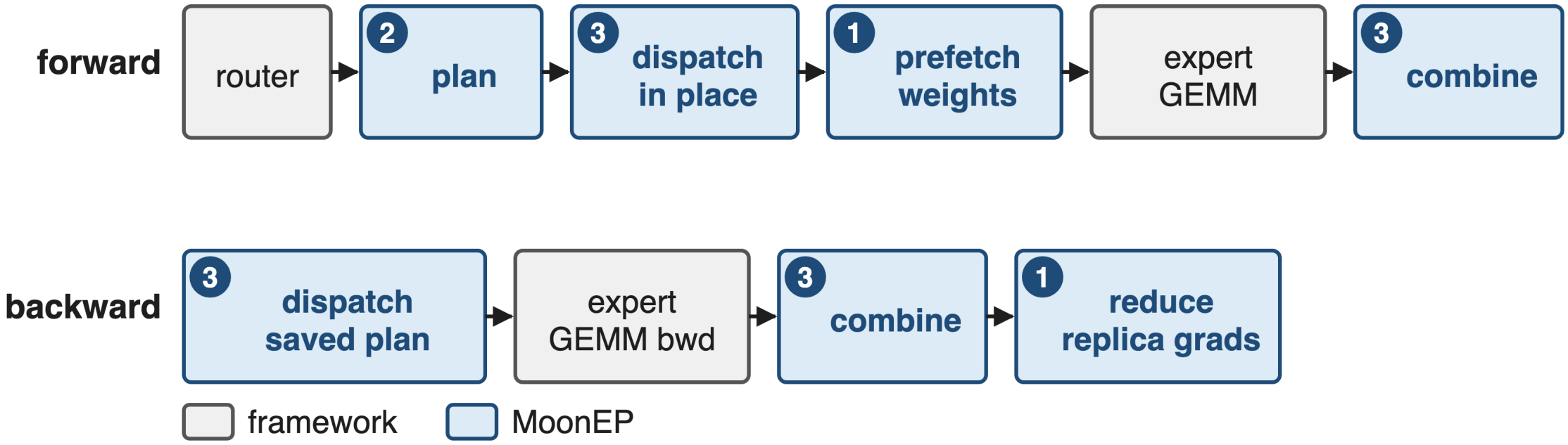
- Fill the emptiest GPU to S×K from the fullest
- ≤ R–1 fills, each from **one** home GPU
- So ≤ E/R replicas; some routings need ≈ $E(R-1)/R^2$



Analytic, E = 384 (K3 report, App. E)

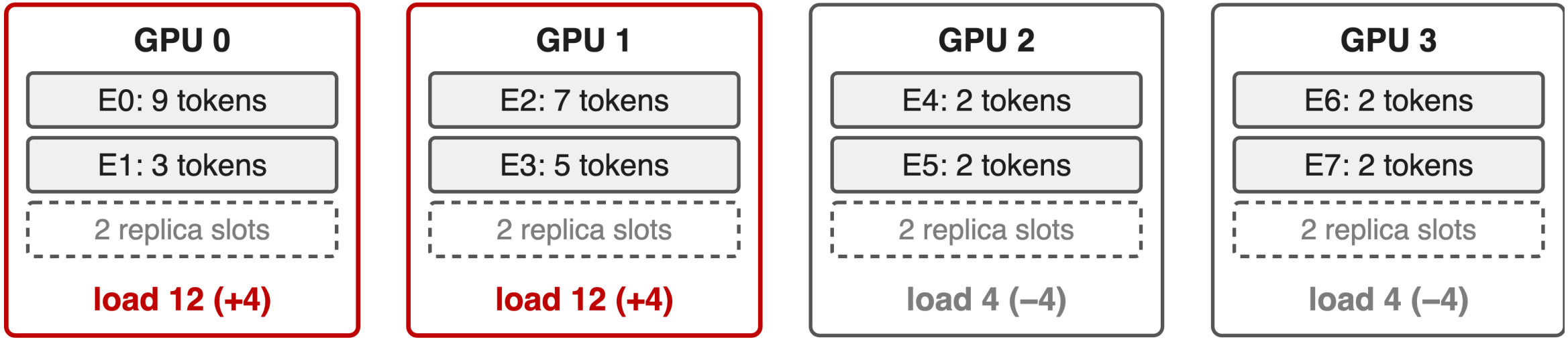
Reserve E/R slots per GPU and a perfect plan always exists

Three mechanisms run inside every MoE layer



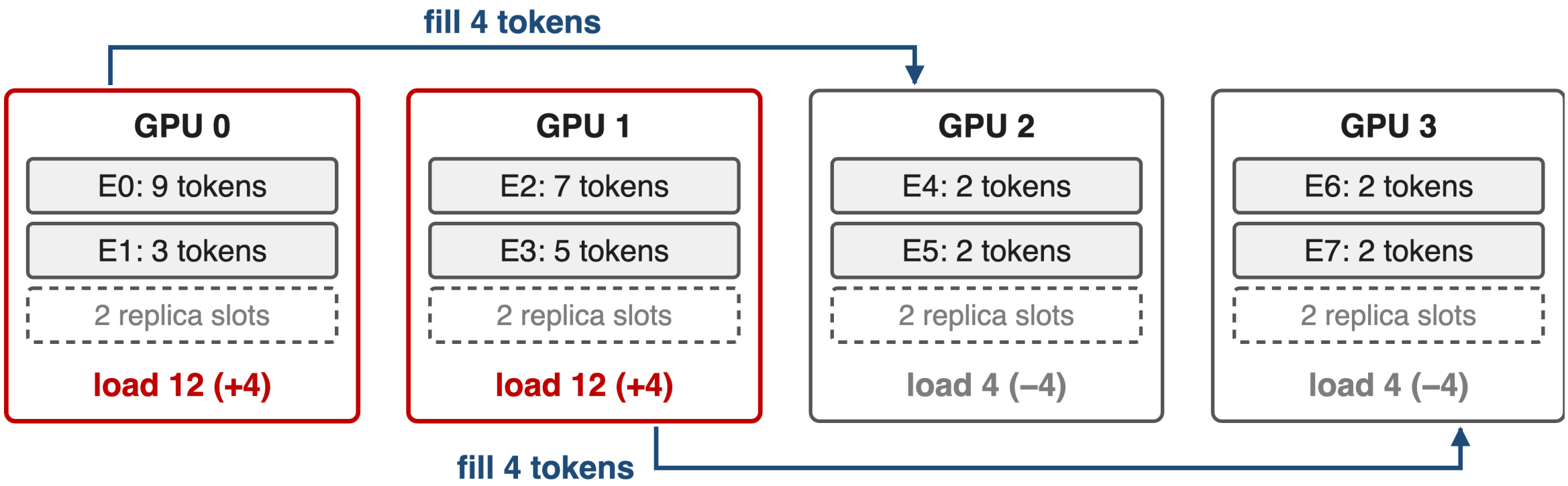
❶ dynamic redundant experts · ❷ online GPU planning · ❸ zero-copy static shapes

❶ Fill the emptiest GPU from the fullest



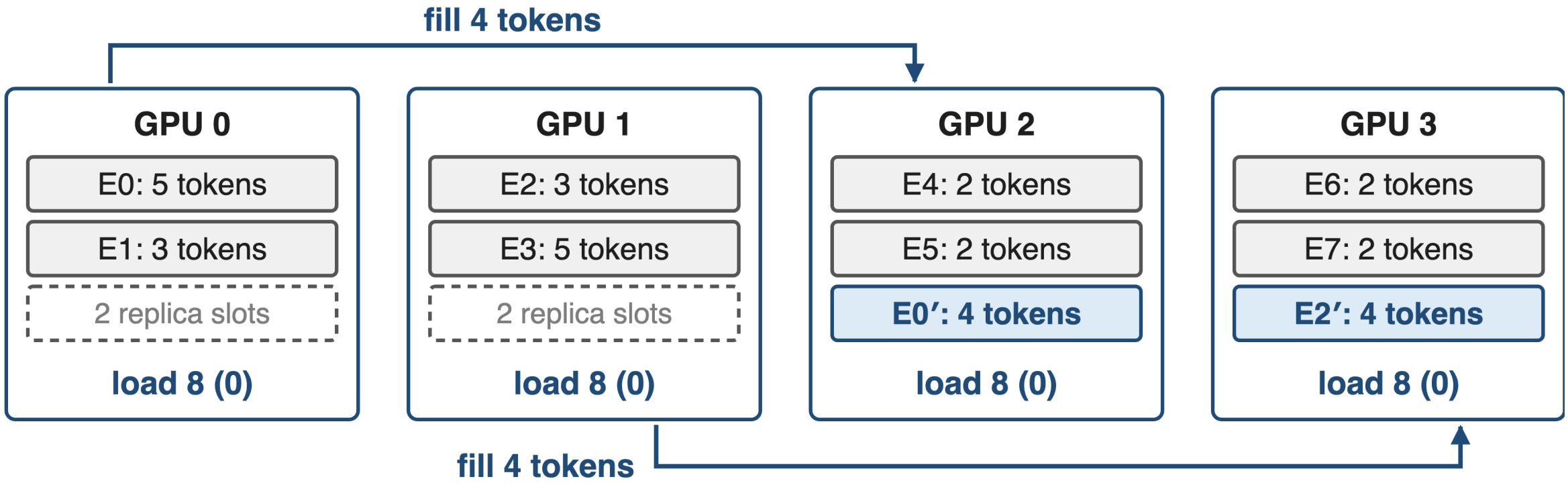
Step 1: balance = load – S×K for each home GPU

❶ Fill the emptiest GPU from the fullest



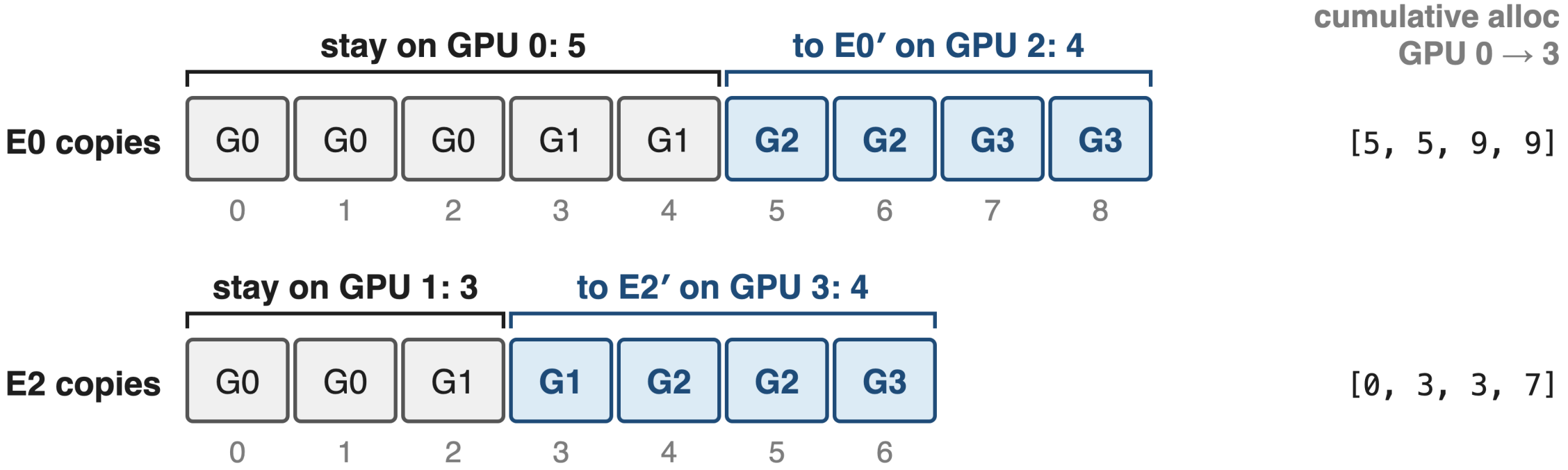
Step 2: fill the emptiest GPU from the fullest, exactly to S×K

❶ Fill the emptiest GPU from the fullest



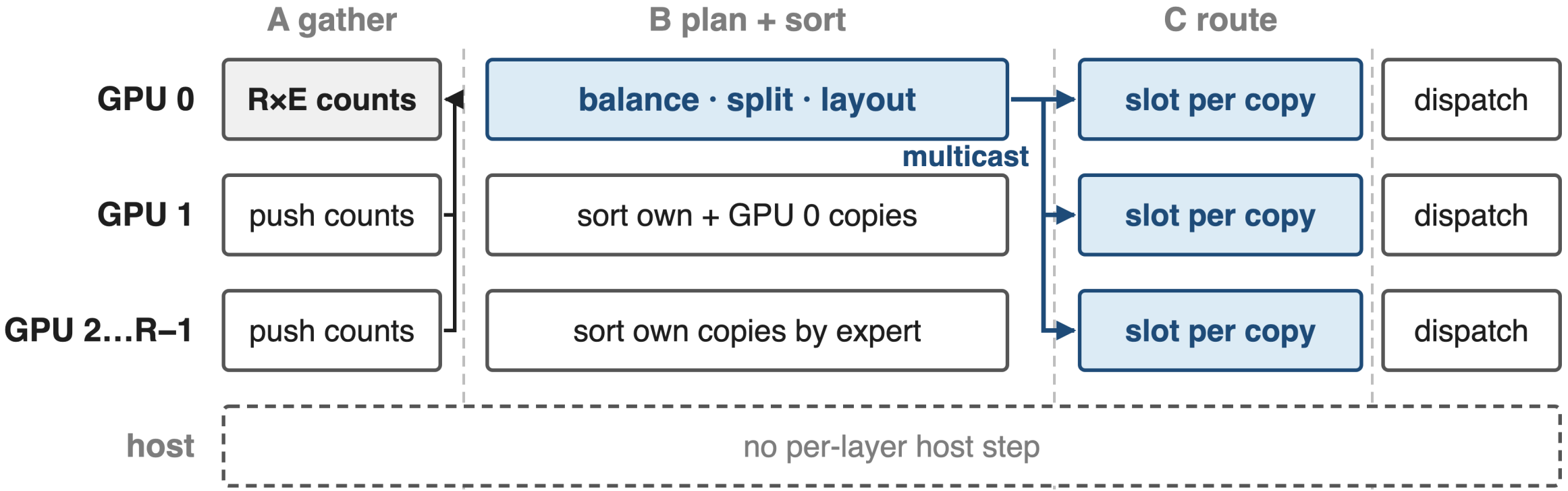
Step 3: take each quota from the home expert with the most tokens left

❶ Expert copies split into contiguous ranges



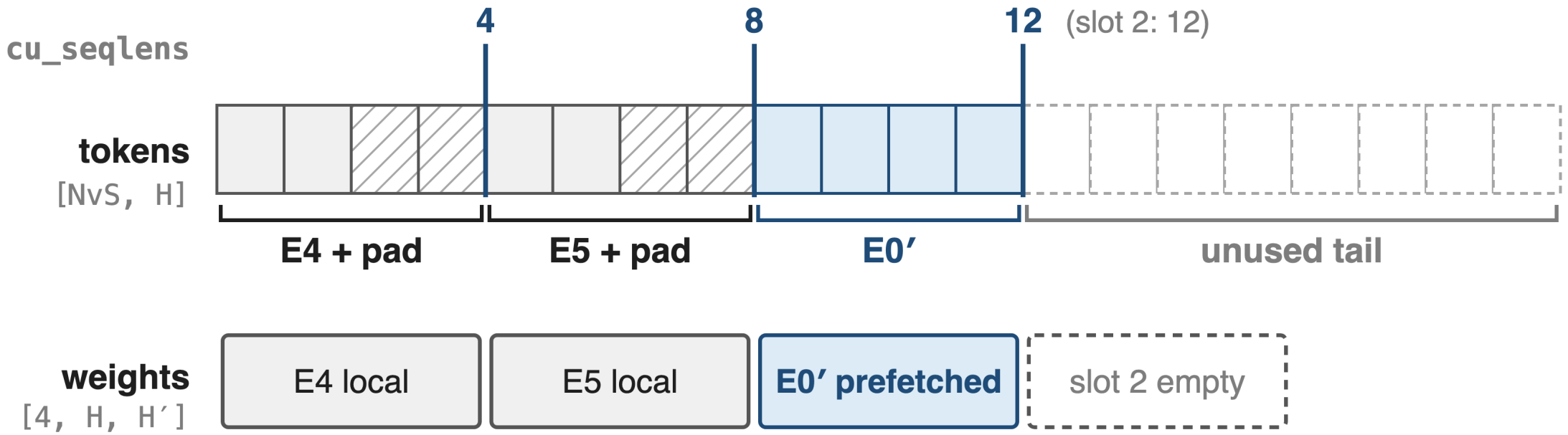
Copy g goes to the first GPU whose cumulative allocation exceeds g

❷ GPU 0 plans while the others sort



One launch per micro-batch and layer: ≈2% of dispatch time (≈45 µs)

❸ Tokens land in a fixed, expert-grouped buffer

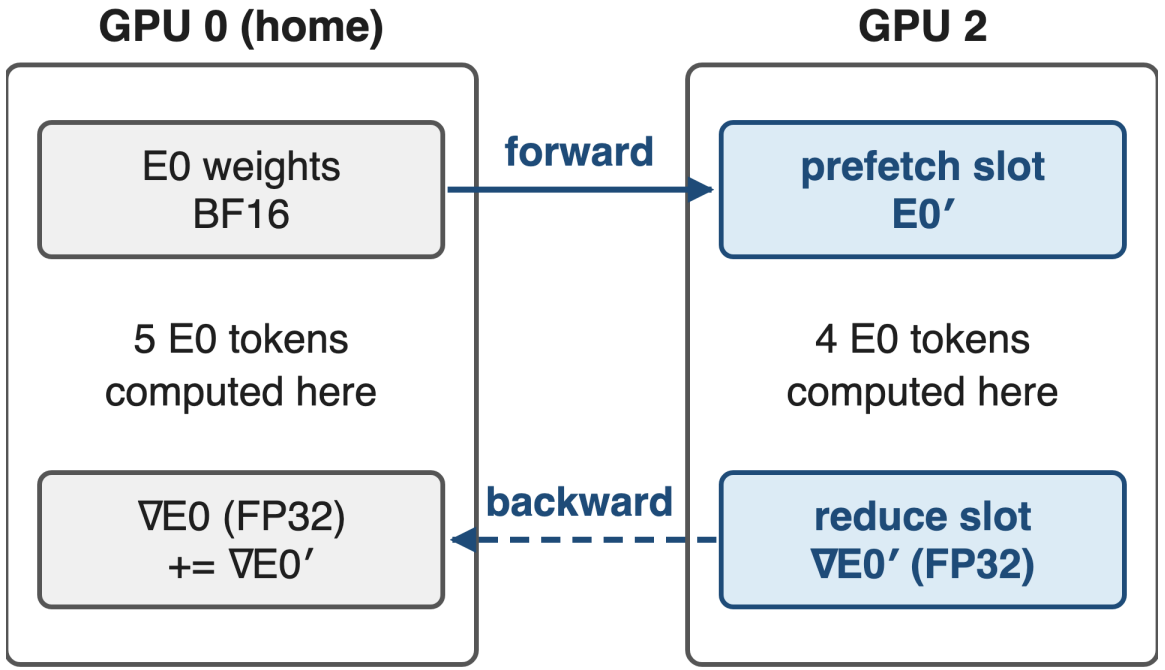


GPU 2 in the example: NvS = S×K + padding slack = 20 rows (P = 4); benchmark: 77,824 rows (P = 128)

The GEMM reads views of this buffer: no permute copy, no host sync

❶ Replicas borrow weights and return FP32 grads

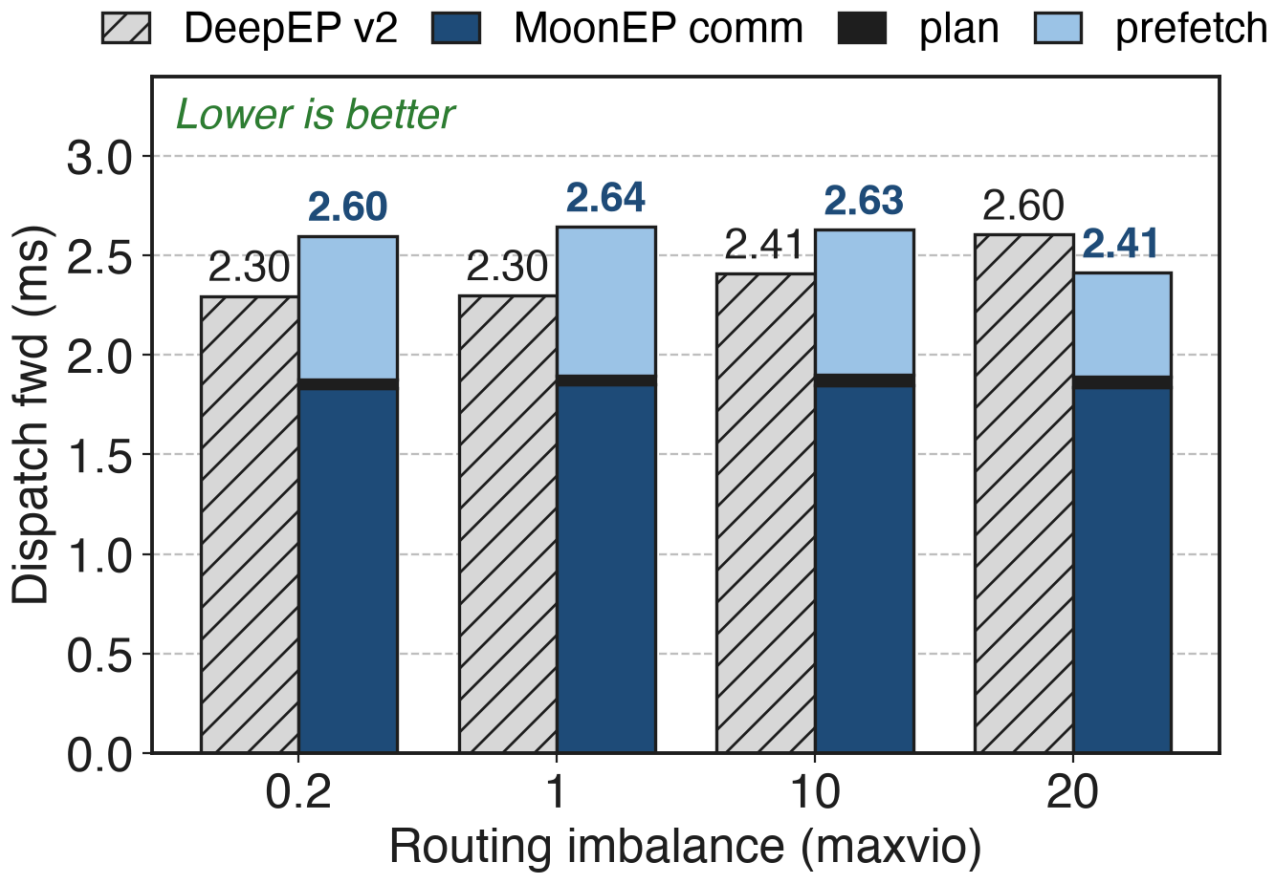
- Forward: home GPU **pushes** BF16 weights
- Backward: home GPU **pulls** FP32 grads, adds them
- Pool: E/R slots per projection, shared by layers



∇E0 = home part + replica part: same update as no replicas

Plan takes ≈2% of dispatch, prefetch up to 29%

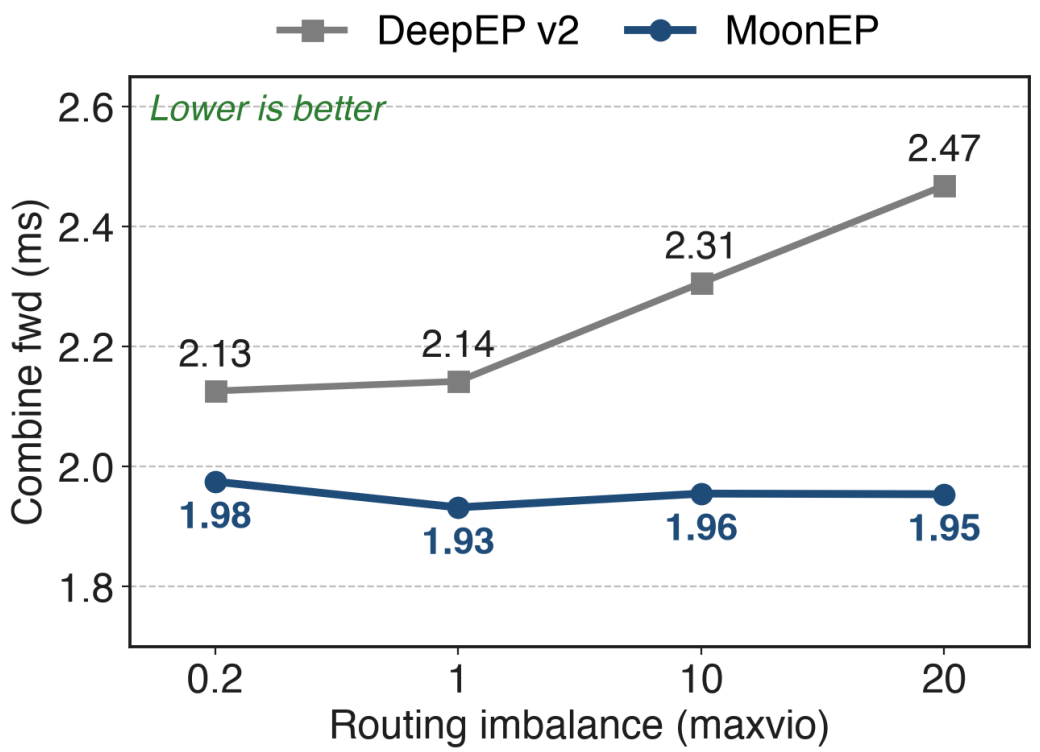
- Plan ≈ 40-50 µs
- Prefetch ≈ 0.5-0.75 ms
- Comm alone ≈ 20-29% below DeepEP v2



Totals: README chart labels; MoonEP segments estimated from that chart

Combine stays flat; DeepEP v2 grows 16%

- MoonEP: 1.93-1.98 ms at every maxvio
- DeepEP v2: +16% from maxvio 0.2 to 20
- No hot GPU sets the pace



–21% combine latency at maxvio 20

Limits: prefetch, pool memory, per-expert skew

- **Prefetch**: 22-29% of dispatch time (estimated)
- **Pool**: 4.2 GB BF16 + 8.5 GB FP32 at EP=8
- **Near-optimal** plan; per-expert segments still vary
- **One NVLink domain**; no RDMA path

Summary

- Offline replicas lag ~10 min; runtime counts force host sync or S×K×R
- **Insight**: one source per receiver, so ≤ E/R replicas suffice
- ❶ fill + split · ❷ one GPU launch · ❸ fixed S×K buffer
- Plan **≈2%** of dispatch; combine flat, **–21%** at maxvio 20

https://github.com/MoonshotAI/MoonEP